future internet

MDPI

Article

# Context-Aware Software-Defined Wireless Networks: An AI-Based Approach to Deal with QoS

**Dainier González Romero [1,2,†], Sergio F. Ochoa [3,*,†] and Rodrigo M. Santos [1,2,*,†]**

[1] Electrical Engineering and Computers Department, Universidad Nacional del Sur, San Andrés 800, Bahía Blanca 8000, Argentina; dainier.gonzalez@cs.uns.edu.ar
[2] Computer Science and Engineering Institute, Universidad Nacional del Sur—CONICET, San Andrés 800, Bahía Blanca 8000, Argentina
[3] Computer Science Department, Universidad de Chile, Beauchef 851, Santiago 837-0456, Chile
* Correspondence: sochoa@dcc.uchile.cl (S.F.O.); ierms@uns.edu.ar (R.M.S.); Tel.: +54-2914595181 (R.M.S.)
† These authors contributed equally to this work.

**Abstract**

Many IoT systems require real-time communication, which imposes strict timing constraints on data transmission and stresses network propagation models. These systems need to address these communication requirements using wireless networks and also manage quality of service. While Software-Defined Wireless Networks (SDWNs) offer a compelling alternative for these scenarios, they lack dynamic mechanisms to autonomously adapt network behavior to fluctuating operational conditions. In order to do that, this paper builds on the authors' previous work and shows how to implement Context-Aware Software-Defined Wireless Networks (CA-SDWNs) that use a self-adapting traffic management strategy to deal with dynamic real-time requirements. In particular, it adapts the medium access protocol parameters to changes in the operational context using an intelligent agent in the control loop of the network. We implement the CA-SDWN model using the NS-3 simulator, and that implementation is made available for researchers and developers through an open-source library. The model is evaluated using several SDWNs that operate under dynamic conditions. The experimental results show how incorporating artificial intelligence into the control loop enables the use of the context information to enhance the predictability of the medium access protocol parameters, thus handling different traffic QoS according to the demand of IoT applications. It represents a clear contribution for researchers and developers of these systems when they have to deal with QoS and real-time constrained communication in SDWNs implemented on WiFi.

**Keywords:** artificial intelligence; context-aware software-defined wireless networks; IoT systems; quality of service; adaptation of network parameters

Check for updates

Academic Editors: Faycal Bouhafs and Sachin Sharma

Received: 10 January 2026
Revised: 12 March 2026
Accepted: 16 March 2026
Published: 19 March 2026

**Copyright:** © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

## 1. Introduction

Systems that support smart environments, usually IoT applications, require processing data to obtain information and acquire knowledge of the current situation in the scenario being monitored or controlled. Based on it, these applications make autonomous decisions or support human beings to make them when needed. To achieve this, data must be transmitted within strict latency bounds and processed under real-time constraints [1]. Precision farming, Industry 4.0/5.0, smart cities, environmental monitoring, disaster relief, and smart health are examples of application domains where this type of communication is usually required [2–6].

The communication infrastructure used by these applications involves mainly stationary nodes located at precise locations, where they measure specific physical variables (sensors) or execute particular control actions (actuators). Although mobile nodes (e.g., unmanned vehicles) may be present, they constitute a minority of the devices in most application domains. This network composition can be inferred from the IoT reference architectures [7–9]. These models show that mobile end-users access the shared data from the cloud or any other infrastructure that does not interfere with the data transmission of the IoT systems.

In these IoT scenarios, managing real-time traffic is a major challenge, as the wireless network should support different quality of service (QoS). Technologies such as 5G have been identified as useful to deal with it through the adoption of modulation techniques and simultaneous multi-user transmissions [10]. In addition, the introduction of multi-numerology in 5G networks has also been shown to improve the QoS for end-users' equipment, as reported in [11]. In particular, this latter work presents a framework that manages diverse types of services with different priorities, based on the allocation and scheduling of the radio frequencies.

In addition, several efforts have been made to introduce the Software-Defined Network (SDN) paradigm within wireless networks based on WiFi [12–14] and 5G [6,11,15]. Most of them are focused on supporting routing from the gateways (i.e., from the intermediate network nodes) to the Internet, without a particular treatment of the wireless links between sensors or actuators and the gateways. These strategies limit the capability of these proposals to provide real-time communication and QoS for IoT applications when required.

In this communication scenario, the SDNs appear as an alternative that could provide real-time communication and various QoS based on IEEE 802.11 networks [16]. This allows implementing several network types, like WMNs, MANETs, VANETs, and FANETs [17]. However, these communication infrastructures, named Software-Defined Wireless Networks (SDWNs), are still in their infancy [5,16]. For instance, they have to deal with an important number of challenges, including network management and support for real-time communication and QoS.

As wireless network hosts change frequently (e.g., due to the device's mobility and usual connection/disconnection process), a dynamic configuration of the network is required. In that scenario, incorporating an intelligent agent allows the network to adapt on the fly the medium-access parameters of the hosts and thus to deal with the different QoS requirements of IoT applications. Therefore, there are various research efforts exploring this opportunity, but to the best of our knowledge, none of them are focused on dealing with dynamic QoS in SDWNs and particularly to support IoT solutions.

It is important to recognize that 5G networks represent the likely future of dynamic SDWNs; however, their operational cost is still too expensive to be adopted by most IoT systems, compared to using WiFi-based solutions.

Due to these reasons, this paper builds on the authors' previous work [18] and proposes a model to implement Context-Aware Software-Defined Wireless Networks (CA-SDWNs) by adding an intelligent agent (IA) to the network controller. This IA includes a machine learning algorithm that predicts the performance of the network according to context variables, and also a Bayesian optimization procedure that determines the most appropriate network parameters to maximize its performance. The context information and the optimized network parameters are used as input by the network controller to dynamically adjust its current parameters and thus to handle different QoS according to the traffic demand of the IoT applications.

CA-SDWNs utilize a context-aware access point controller (CAP controller) to manage traffic in accordance with the IEEE 802.11 standard's access point specification [19]. This

proposal is evaluated using several SDWN scenarios that consider different numbers of nodes, deployment layouts, and transmission demands. To perform that evaluation, the scenarios and the CAP controller are simulated using NS-3.

The results show how the implementation of the CAP controller allows for the improvement of different performance indicators (like throughput, delay, and packet loss ratio) in comparison to the use of both a regular wireless network (without an SDN controller) or a regular SDWN based on IEEE 802.11 (i.e., with a controller that uses default parameters). However, the main benefit of using the CAP controller is the capability that the network gains to self-adapt its behavior, according to fluctuations in the QoS demanded by IoT applications. This feature becomes the SDWNs in context-aware networks in terms of QoS management.

To facilitate reuse by the research community, the NS-3 components and libraries developed as part of this work have been published in an open GitHub repository [20], as well as the dataset with the evaluation results of the CA-SDWN [21].

The next section provides a review of recent literature on Software-Defined Wireless Networks and also the background on IEEE 802.11e medium access control. The latter helps understand the implementation of CA-SDWNs. Then, Section 3 presents the CAP controller, which is the main component required to implement these networks. Section 4 describes the evaluation process of the network performance, and Section 5 discusses the experimental results. Finally, Section 6 presents the conclusions and future work. Appendix A describes the implementation and configuration of CA-SDWNs using the NS-3 simulator.

## 2. Related Work

SDNs have transformed traditional network management by decoupling the control and data planes, enabling centralized programmability through logically centralized controllers. While initially applied to wired networks using protocols like OpenFlow [22], extending SDNs to wireless environments introduced new challenges, including channel variability, energy constraints, and Real-Time Quality of Service (RT-QoS) requirements. These challenges become more pronounced in WSNs, composed of devices with resource limitations [2], giving rise to Software-Defined Wireless Sensor Networks [23], which aim to provide dynamic configurability, efficient resource management, and traffic prioritization.

Next, we present and characterize the main proposals that allow the implementation of SDWNs and also analyze their capabilities to deal with real-time communication restrictions. Then, we explain how IEEE 802.11e networks can be used to support the implementation of SDWNs and particularly CA-SDWNs.

### *2.1. Overview of SDWNs*

An early effort in this domain was ÆtherFlow [24], which adapted the OpenFlow protocol to wireless scenarios through the TinyNBI abstraction, introducing programmable control over parameters like transmission power and SSID configuration. Evaluated using commercial access points, it showed improved performance in Layer 2 handoff scenarios. However, ÆtherFlow was tightly bound to a centralized architecture, lacked formal QoS abstractions, and did not support distributed control, limiting its scalability.

To address scalability challenges, the WNOS (Wireless Network Operating System) [25] introduces a distributed approach tailored to infrastructure-less networks, such as IoT, ad hoc, and sensor systems. By decoupling high-level policy design from low-level device execution, WNOS compiles centralized programs into distributed, cross-layer logic that runs locally on each node. Its architecture comprises a framework that provide for abstraction, a decomposition engine, and a programmable protocol stack. It has demon-

strated notable performance improvements as well as flexibility and scalability, all without requiring global state dissemination.

Building on these ideas, Po-Fi [26] leverages the Protocol Oblivious Forwarding (POF) to unify IEEE 802.3 and 802.11 under a common flow model. Po-Fi transforms access points into programmable entities capable of dynamic wireless management, introducing a centralized Time-Division Multiple-Access mechanism to mitigate interference issues common in 802.11 networks. Furthermore, decentralizing tasks (e.g., signal analysis) helps offload the central controller, improving scalability and adaptability.

As network density increases, particularly in IoT contexts, newer architectures like DENIS-SDN [27] introduce network slicing to enhance performance. Through logical and physical slicing, it optimizes routing and interference management while incorporating tools such as CODET and a dashboard. Evaluations showed significant improvements in packet delivery, especially with physical slicing, demonstrating effectiveness in ultra-dense deployments.

Earlier frameworks, such as Sensor OpenFlow (SOF) [23], laid the foundation by introducing OpenFlow-based programmability in WSNs. With support for compact and attribute-value addressing and mechanisms such as Control-Message Quenching to reduce signaling overhead, SOF enhanced flexibility. However, it lacked integrated QoS prioritization and mechanisms for ultra-density or slicing, making it less suitable for modern IoT demands.

SDN-WISE [28] further refined this idea with a design customized to WSNs. Using stateful forwarding and finite-state-machine modeling, it enabled nodes to operate semi-autonomously, reducing controller dependency. It also introduced energy-saving features like duty cycling. Nonetheless, SDN-WISE did not formally incorporate QoS or address scenarios with high mobility or interference, challenges that newer systems like DENIS-SDN have begun to tackle.

From a research-oriented perspective, IT-SDN [29] offers a modular framework for evaluating SDN control algorithms on Contiki-based devices. Although flexible and suitable for experimentation, it lacks native support for QoS, automatic energy-saving mechanisms, and integrated security, limiting its applicability to real-world deployments.

Complementing architectural advances, some works focus on specific optimization techniques. Karakurt and Koçak [30] proposed a fuzzy logic-based controller to dynamically tune the parameters of the Carrier Sense Multiple Access with Collision Avoidance (e.g., RSTV, FTV, buffer size) based on real-time feedback, achieving significant gains in throughput and delay. That work highlights the importance of adaptive mechanisms in highly dynamic environments.

Similarly, Fondo-Ferreiro et al. [31] developed an SDN-based system for fast and efficient access-point reassignment in dense wireless environments. Using multi-layer information and predictive modeling, their approach achieved low-latency handovers and improved QoS, though it stopped short of full cross-layer optimization.

The architecture of the controller remains a central issue in SDWN's design. Centralized models (e.g., ÆtherFlow and Po-Fi) offer simplified management but suffer from latency and fault tolerance issues. Conversely, distributed designs (e.g., SDN-WISE) provide resilience but introduce synchronization complexity. Hybrid models, such as logically centralized but physically distributed controllers, balance these trade-offs. For example, Heller et al. [32] demonstrated that multiple controllers improved fault tolerance, though with diminishing returns. More recently, Mudvari and Tassiulas [33] applied Deep Reinforcement Learning to dynamically optimize controller placement, reducing synchronization overhead and improving load balancing at the cost of increased computational effort.

A conceptual advance in this area was presented by the authors in [18], who proposed an SDWN architecture tailored for real-time IoT applications. By leveraging POF's flexibility, that work introduced a traffic classification system with three levels: strict, adaptive, and non-real-time. Moreover, that proposal introduced mechanisms to support up to 23 QoS levels via IEEE 802.11 fields. Although the model remained theoretical, it bridged the gap between conceptual frameworks and practical constraints, laying the foundations to implement time-sensitive wireless applications.

In this sense, technologies like 5G have been identified as useful to deal with it. In fact, advances in communication theory have shaped both 5G technology and WiFi standards, particularly through the adoption of modulation techniques and simultaneous multi-user transmissions [10]. Rather than being a linear transfer from 5G to WiFi, these developments reflect a complementary and convergent evolution, where cellular and WiFi technologies integrate multi-user architectures, advanced modulation (e.g., OFDMA, 1024-QAM), and scheduling mechanisms to ensure efficiency, low latency, and high throughput in diverse deployment scenarios [10,34,35].

In [36], the authors discuss the use of 5G technology with virtualization and SDN controllers to support IoT applications. The SDN enables the programming of the network to enhance the performance of a group of network devices. However, this approach is not oriented to supporting scheduling or classification of traffic in wireless local area networks. Additionally, in [37], the authors analyze the scalability of IoT systems considering 5G technology and conclude that it is not suitable to support highly dense networks (with at least one million devices per $km^2$). To address these scenarios, the authors propose a 6G system architecture inspired by the concept of the sharing economy, which uses the concept of spectrum programming. The positive results obtained in the simulations open several opportunities to use this proposal as a basis to implement SDNs.

In [15], an adaptive routing strategy is proposed for resource optimization in 5G core networks using SDN technologies. The proposal is focused on both managing the main network and how to optimize the use of resources available for end-users. The SDN is not oriented towards managing the wireless links, in this case, between mobile nodes and base stations from which the traffic is routed. Similarly, in [12], the authors propose a packet-scheduling method based on AI that operates at switches and routers of an SDWSN, and it does not address the wireless traffic between the nodes and the gateways; therefore, that proposal has the above-mentioned limitations when managing real-time communication and QoS.

In [38], the authors present a congestion control algorithm for 5G networks based on SDNs. The proposal is oriented towards controlling the switches and routers using the OpenFlow and OpenConfig protocols, i.e., the congestion is controlled from the wired network and not the wireless links. This approach limits the suitability of this proposal to provide real-time communication and QoS support in the stated communication scenario. Similar approaches are reported in [14,39], where the SDN controller handles the wired network and not the wireless traffic.

In [13], the authors introduce a testbed for smart IoT applications based on SDWSNs. Although interesting, the paper does not describe how the network works or the protocols used to implement it.

In contrast to previous contributions, the work reported in this article operationalizes the model proposed in [18] by integrating IEEE 802.11e's native QoS mechanisms with the protocol-agnostic flexibility of POF, supported by an embedded intelligent agent. This approach enables effective traffic differentiation, real-time prioritization, and adaptive control. Unlike Po-Fi [26], which omits native 802.11e support, the proposal provides a functional, open-source implementation in NS-3, thereby promoting reproducibility and

real-world applicability. Tables 1 and 2 compare these SDWN approaches, covering their main architectural aspects, implementation characteristics, strengths, and limitations for real-time communication.

Despite an extensive literature review, to the best of the authors' knowledge, the development of SDNs for wireless interfaces remains limited. While the standard OpenFlow implementation for wired networks is not directly transferable to wireless environments due to architectural and mobility constraints, its core principles have inspired various adaptations such as AetherFlow [24], Sensor OpenFlow [23], and others. These efforts have demonstrated increasing flexibility and control, paving the way for intelligent, context-aware SDWN systems such as the one proposed in this article.

**Table 1.** Summary of approaches and architectural characteristics of SDWN proposals.

| Proposal (Reference) Year | Target Environment | Approach/Core Idea | Control Paradigm | QoS Support |
|---|---|---|---|---|
| ÆtherFlow 2015 [24] | Enterprise WiFi | OpenFlow extension for wireless control | Centralized | No |
| WNOS 2021 [25] | Wireless Mesh/Ad hoc | Distributed SDWN via optimization engine | Logical centralized (distributed execution) | Implicit (policies) |
| Po-Fi 2021 [26] | Enterprise WiFi | POF-based programmable APs | Centralized | Partial (TDMA) |
| DENIS-SDN 2023 [27] | Ultra-dense IoT | Network slicing (logical/physical) | Centralized (OpenFlow) | Yes |
| SOF 2012 [23] | WSN | Lightweight OpenFlow stack | Centralized | Basic (no priorities) |
| SDN-WISE 2015 [28] | WSN | FSM-based stateful SDN | Centralized + local logic | No |
| IT-SDN 2017 [29] | WSN/IoT | Open modular simulation framework | Centralized (external PC) | No |
| Fuzzy CSMA/CA 2023 [30] | WLAN | Fuzzy logic for adaptive MAC | Local (device feedback) | Yes |
| Fast Handover QoS 2019 [31] | WiFi/5G | Predictive AP reassignment | Centralized SDN | Yes |
| Controller Placement 2012 [32] | WAN/Generic | Placement via k-center/k-median | Centralized | Indirect |
| Sync + Placement 2024 [33] | Ad hoc WAN | DRL-based joint optimization | Distributed | Indirect |
| Llinás et al., 2023 [18] | IoT/Real-time WiFi | POF + 802.11e priority classes | Centralized | Yes (802.11e + custom) |
| Al-Hamid et al., 2023 [13] | IoT/WSN | SDWSN virtualization testbed | Centralized (conceptual) | No |
| Alzahrani & Fotiou 2021 [14] | IoT Security | SDN to secure group comms. | Centralized (POX) | Yes (via auth.) |
| Ellawindy & Heydari 2021 [39] | SD-WAN/CDN | Crowdsourced QoE for SDN routing | Centralized (ONOS) | Indirect (QoE feedback) |
| Yang & Tsai 2024 [38] | 5G Core | Bandwidth Reservation Fairness (BRF) | Centralized (OpenFlow) | Yes |
| Coêlho et al., 2024 [36] | 5G IoT (survey) | SDN/NFV review for slicing/security | N/A | N/A |
| Xu 2025 [15] | 5G Core | NFV-SDN migration + routing | Centralized | Indirect (latency) |
| This work 2026 | Real-time IoT over WiFi | POF + 802.11e SDWN + AI | Centralized | Yes (EDCA/TXOP) |

**Table 2.** Summary of implementations, strengths, and limitations of SDWN proposals.

| Proposal Year | Implementation/Validation | Key Strengths | Primary Limitations |
|---|---|---|---|
| ÆtherFlow 2015 [24] | Testbed (APs + station) | Programmable WiFi parameters; reduced L2 handoff delay | Lacks scalability; no QoS abstraction or hierarchy |
| WNOS 2021 [25] | Testbed (21 USRPs) | High scalability; cross-layer optimization | High complexity; not compatible with OpenFlow/802.11 |
| Po-Fi 2021 [26] | Testbed (commercial APs + POF) | Unified wired/wireless pipeline; flexible processing | No native 802.11e QoS |
| DENIS-SDN 2023 [27] | Simulations + dashboard | Improves delivery rate; traffic prioritization | Static controller placement; bottleneck risk |
| SOF 2012 [23] | Simulation + PoC | Early SDN-WSN proof; CMQ for efficiency | No slicing, QoS, or security; limited scalability |
| SDN-WISE 2015 [28] | Testbed (<100 ms resp.) | Energy-efficient; stateful rules; visualization via WISE-Visor | No hierarchical control; not validated under mobility |
| IT-SDN 2017 [29] | COOJA + Contiki + TelosB | Flexible for ND/CD evaluation; real deployment support | No QoS or security automation; research-focused |

**Table 2.** *Cont.*

| Proposal Year | Implementation/Validation | Key Strengths | Primary Limitations |
|---|---|---|---|
| Fuzzy CSMA/CA 2023 [30] | Simulation | Better throughput and delay; adaptive | Not integrated with SDN stack; scope limited to MAC |
| Fast Handover QoS 2019 [31] | Trace-driven testbed | Seamless handover; context-aware (QoS) | Not generalizable; no cross-layer design |
| Controller Placement 2012 [32] | Simulation | Foundational model; latency/fault trade-off | No wireless-specific perspective |
| Sync + Placement 2024 [33] | Simulation | Adaptive to topology changes; optimizes sync and placement | High computational cost; no hardware validation |
| Llinás et al. 2023 [18] | Conceptual (Mininet planned) | Fine-grained QoS; protocol-agnostic | No real validation; lacks benchmarks |
| Al-Hamid et al. 2023 [13] | Contiki Cooja + TI hardware | Enables pre-deployment validation; OAD reconfigurability | No explicit SDN protocol; vague controller role |
| Alzahrani & Fotiou 2021 [14] | Mininet + Open vSwitch | Prevents false resource announcements; WoT-TD based | No control over last-mile wireless; limited to NAPs |
| Ellawindy & Heydari 2021 [39] | Mininet | Real-time QoE routing improvements | Focuses only on wired SD-WAN; no wireless control |
| Yang & Tsai 2024 [38] | Mininet simulations | Fast convergence; fairness in bandwidth allocation | Oriented to wired core links, not wireless edges |
| Coêlho et al. 2024 [36] | Survey (literature-based) | Identifies gaps; role of slicing for IoT | Not a proposal; no implementation |
| Xu 2025 [15] | Simulation (200–500 nodes) | Dynamic VNF migration; latency reduction | Focus on core VNFs; no wireless control |
| This work 2026 | NS-3 (open-source) | Real-time QoS enforcement; reproducible; extends Po-Fi; manages dynamic traffic demand | Centralized control scalability remains a challenge |

### *2.2. Overview of the IEEE 802.11e Medium Access Control*

Unlike in wired networks, wireless communication presents several challenges that affect QoS implementation, including lower transmission channel reliability, variability in signal quality, and limitations in available spectrum [40]. Wireless networks are inherently more susceptible to interference and packet loss, negatively affecting service quality. Furthermore, optimizing power consumption is crucial in wireless networks, especially for IoT devices and portable equipment that rely on batteries, a limitation not typically found in wired networks [41–43].

To ensure efficient data transmission and mitigate these limitations, wireless networks employ specific medium access mechanisms. The most widely used is Carrier Sense Multiple Access with Collision Avoidance, which allows devices to assess whether the channel is occupied before transmitting, preventing data collisions [44] by reserving the use of this before actually transmitting the message.

The IEEE 802.11e standard was developed as an extension to provide QoS mechanisms in wireless networks, optimizing data transmission based on the priority of each traffic flow. It introduces two key methods for managing channel access: Enhanced Distributed Channel Access (EDCA) and HCF Controlled Channel Access (HCCA) [45].

EDCA categorizes traffic into distinct access categories (ACs), assigning a specific priority to each. To manage this prioritization, EDCA defines specific parameters such as CWmin and CWmax, which establish the minimum and maximum contention windows, respectively, as well as the Arbitration Inter-Frame Space (AIFS) and the Transmission Opportunity (Txop). The parameters CWmin and CWmax determine the waiting time before a device can access the channel, while the AIFS and Txop regulate the channel access time for different types of traffic [46,47]. For example, voice traffic requires low latency, has a shorter AIFS and a smaller contention window, allowing it to access the channel preferentially. Figure 1 illustrates the EDCA mechanism.

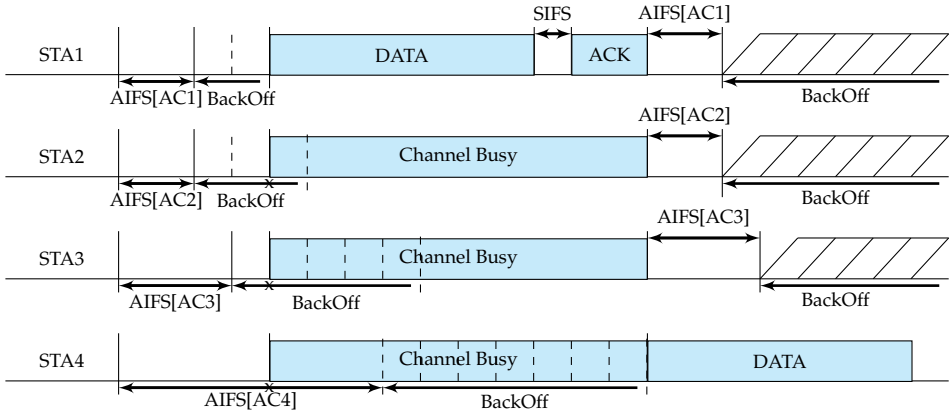



**Figure 1.** Channel access in EDCA.

On the other hand, HCCA uses a centralized control unit that assigns specific channel access times to each device, allowing better traffic management and ensuring QoS for real-time applications [45]. This approach is particularly useful in high-density environments and where traffic load is predictable, such as in videoconferencing applications or critical data transmission.

The IEEE 802.11e standard classifies network traffic into four access categories to manage QoS. Table 3 presents the parameters for their operation [48]. In particular, the parameters AIFSN, CWmin, and CWmax are not expressed in absolute time units (ms or µs). Instead, they are multiples of the slot time defined by the specific IEEE 802.11e PHY. Next, we introduce these access categories:

1. VO (Voice): This category is assigned to voice traffic, such as VoIP calls, which require low latency and maximum priority to ensure communication quality.
2. VI (Video): Real-time video traffic, such as videoconferencing streams or security cameras, is prioritized to maintain continuous and uninterrupted transmission.
3. BE (Best-Effort): General application traffic, such as web browsing, falls into this category. Although it is not a high priority, BE traffic receives adequate service depending on the available bandwidth.
4. BK (Background): This category is reserved for low-importance or background tasks, such as data synchronization, which can tolerate higher delays without affecting the user experience.

**Table 3.** Summary of EDCA parameters by access category.

| AC Name | AC Description | AIFSN | CWmin | CWmax | Max Txop [ms] | Priority |
|---|---|---|---|---|---|---|
| AC_VO | Voice | 2 | 3 | 7 | 1.504 | Highest |
| AC_VI | Video | 2 | 7 | 15 | 3.008 | High |
| AC_BE | Best-Effort | 3 | 15 | 1023 | 0 | Low |
| AC_BK | Background | 7 | 15 | 1023 | 0 | Lowest |

Although IEEE 802.11e was proposed in 2005, its implementation and widespread use have not been fully generalized. One possible reason is that the standard operates at the data link layer, limiting its ability to influence traffic differentiation at higher layers, such as the network layer. In addition, concerns related to network neutrality may restrict the enforcement of traffic prioritization policies beyond the local scope of the wireless link.

The possibility of using SDN concepts in the wireless realm turns the use of this standard into a true option, as the access points can interact with the Internet, providing traffic with priorities, even keeping the neutrality principle, since the classification is provided by the link layer access points. To complete that, we propose the use of context-aware access points (Po-Fi Access Points). The design and operation of these components are discussed in the next section.

## 3. Description of the Context-Aware Access Point Controller

Consistent with [18], we propose using a POF-based software component (named the *Po-Fi Access Point*) to manage the traffic in SDWNs. Different client devices participating in the IoT system (e.g., regular sensors, weather stations, and actuators) register with a Po-Fi Access Point to be part of the network (step labeled as (1) in Figure 2).

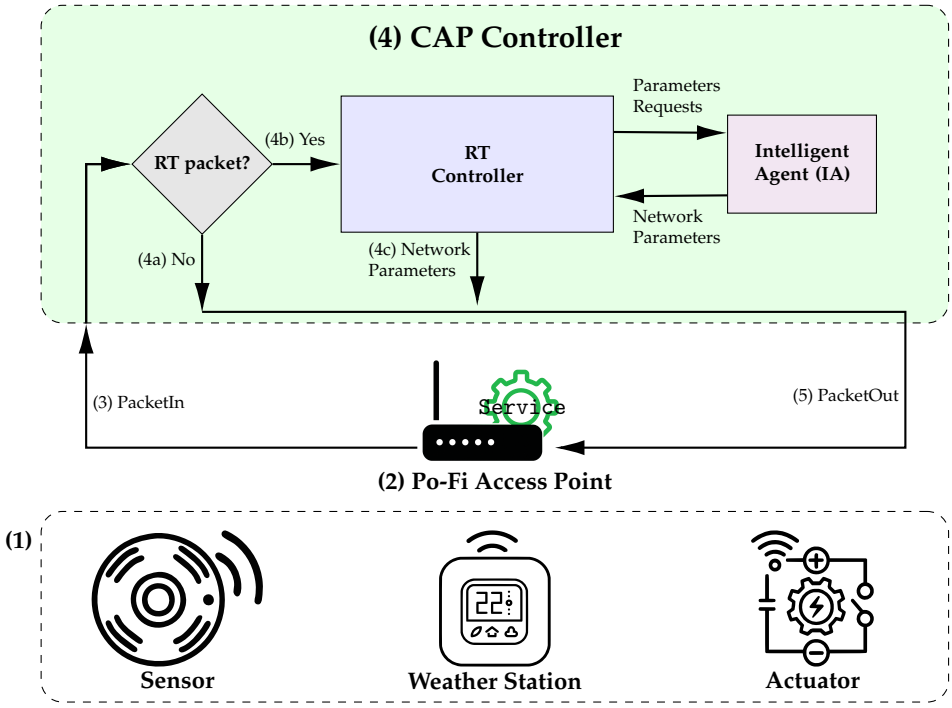



**Figure 2.** Work Scenario of a Po-Fi Access Point.

In every Po-Fi Access Point runs a software service that implements a packet processing pipeline (step (2)). This service maintains tables with context information about the current network status, e.g., number of nodes, type of traffic to handle, and a set of recommendations to manage the traffic considering packet priority. Using that service, the flows incoming to the Po-Fi Access Point pass through a series of tables, where filters and actions are applied based on the fields of each incoming frame and the context information. These actions can be performed at the link, network, or transport layer, depending on the filter that is used.

After processing, the packets are either forwarded to an output port or discarded. If no matching action is found in the tables of the Po-Fi Access Point (i.e., the access point has no information on how to manage the packet), the frame is sent to the CAP controller (context-aware access point controller) via a *PacketIn* message (step (3)). The latter processes the packet and then recommends to the Po-Fi Access Point how to manage that traffic (step (4)).

The CAP controller has one more stage that separates the processing of message flow depending on whether the packet delivery involves time restrictions (i.e., real-time packets). Unrestricted flow is passed, with a lower priority, to the regular pipeline implemented

by the software service of the access point (labeled as 4a). In other cases, the flow is processed by the *RT controller* that considers the QoS according to the message flow type (labeled as 4b).

In that case, the RT-controller provides information to the Po-Fi Access Point, and also recommendations to update the local tables to properly manage the incoming packets (step (5)). Unlike the proposal reported in [26], in this case, the CAP controller includes an *intelligent agent (IA)* that is invoked when there are changes in the network context, e.g., in the number of nodes or priority of the traffic to be handled. In other cases, the software service running in the Po-Fi Access Point decides how to manage that traffic based on the local context information and generates a new recommendation.

Implementing this method of operation is feasible because POF allows packets to be processed without relying on specific protocols. Instead of using fixed IDs for header fields, it extracts information by identifying bit offsets and lengths within the packet (<offset, length>), thus allowing greater flexibility in traffic forwarding [49]. The following subsections detail the priority scheme, the packet processing logic within the Po-Fi Access Point, and the integrated RT controller and intelligent agent.

### 3.1. Packet Priority Scheme

The Po-Fi Access Point organizes these packets into three priority queues: high, medium, and low, ensuring that packets of highest priority are transmitted first. Within the low-priority queue, low packets have precedence over non-real-time packets. This management strategy is performed at the link layer to respect the principle of network neutrality.

To implement this priority scheme, the Po-Fi Access Point takes advantage of the POF capability to extract information; in this case, from the QoS Control field of the MAC header in IEEE 802.11e (Figure 3), where the Traffic Identifier (TiD) and Txop limit fields are specified. In particular, if the field related to Address 4 (optional field) is not used, then the TiD is identified at <192, 3> and the Txop limit at <199, 8>, which are the default values in IEEE 802.11e. Otherwise, they must be shifted by 48 bits.

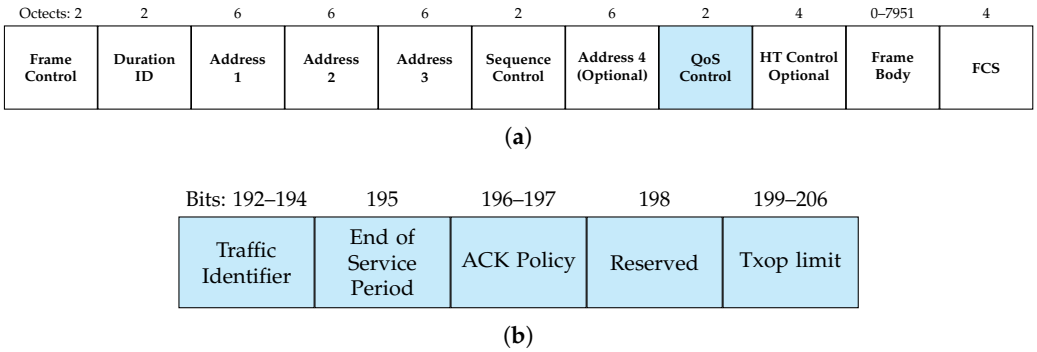



**Figure 3.** MAC layer structure: (**a**) MAC header, (**b**) QoS control.

Packets belonging to the VO (voice) access category are assigned the highest priority, while VI (video) packets are given medium priority. BE (best-effort) packets are considered low priority, whereas BK (background) packets correspond to non-real-time traffic, i.e., they are transmitted whenever there is idle time or in the same way but in the background.

### 3.2. Packet Processing at the Po-Fi Access Point

The allocation of packets to the queues is done by the software service running in the Po-Fi Access Point, according to the traffic management recommendations given by the CAP controller. For this purpose, the message flow is sent between both components, following a typical SDN scheme based on POF.

As shown in Figure 4, when a Po-Fi Access Point receives a packet, the service extracts the values of the QoS control field and checks if there is a previous recommendation to manage the traffic in its POF forwarding tables (step (1)). If there is no recommendation already defined, the access point sends a packet input message (i.e., PacketIn) to the CAP controller. Then, the controller decides how to process the packet and responds with a flow modulation message (i.e., PacketOut) that sets the values for prioritization, queuing, and bandwidth limit modification.

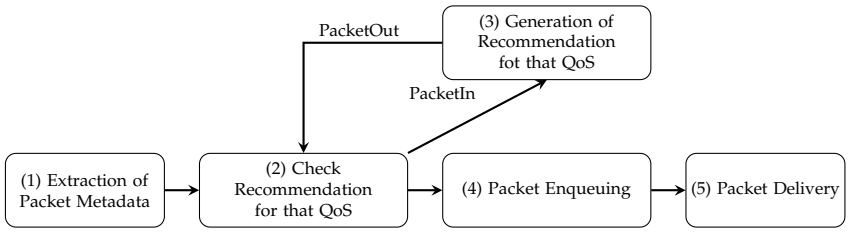



**Figure 4.** Packet processing at the Po-Fi Access Point.

The packet forwarding recommendations consider several scenarios; for instance, if two packets with the same priority arrive (e.g., high–high or low–low), the Po-Fi Access Point transmits one first and places the other in the respective queue until the sending process of the first one has been accomplished.

Instead, if the two packets correspond to the low and non-real-time priority (e.g., low-non-real-time), the low packet is transmitted first, while the non-real-time packet is queued. In case packets from different access categories and different priorities arrive (e.g., high–non-real-time, medium–high, or medium–low), the one with the highest priority is transmitted first, ensuring that delay-sensitive traffic is not interrupted by best-effort or background flows.

When a new packet arrives while others are in the queue, its TiD is compared with those already waiting. If the new packet has a higher TiD, it is sent before the others, regardless of whether it arrived first or later. This dynamic reordering logic in the queue allows the highest priority packets to always be processed first, regardless of when they enter the network.

The Po-Fi Access Point also introduces a dynamic mechanism for adjusting the Txop limit according to the traffic priority supported by the CAP controller. Through a flow modulation message, the access point modifies the Txop limit value (<199, 8>), allocating more or less channel usage time to high-priority packets according to the demand of the IoT system. In this way, high- and medium-rate packets can take advantage of transmission time to reduce latency and improve QoS in real-time applications.

This programmable POF-based approach provides efficient traffic management in WiFi networks. In particular, the capability of the Po-Fi Access Point to extract information from the QoS Control field, assign priority queues, and dynamically modify the Txop limit enables bandwidth usage optimization and improved QoS for real-time IoT systems. Figure 5 shows a UML sequence diagram with the message exchange between two new clients and a Po-Fi Access Point to register the end-devices into the network. In particular, Client 1 is a wireless device that requires non-real-time communication, and Client 2 involves real-time traffic.

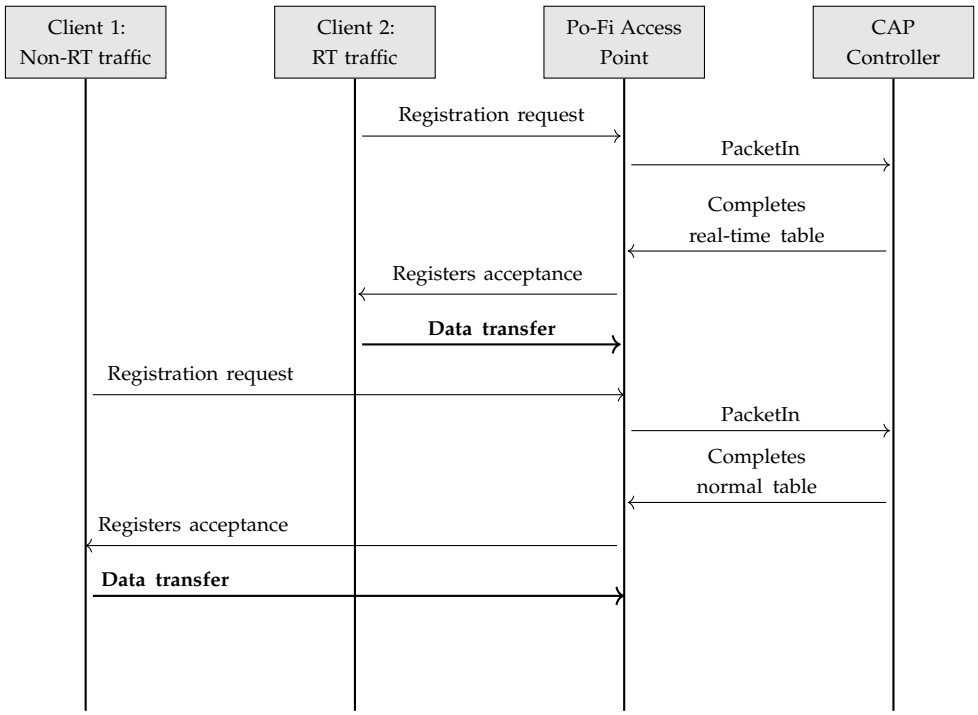



**Figure 5.** Registration process for real-time and non-real-time clients in a Po-Fi Access Point.

### 3.3. Real-Time Controller

Different traffic types require setting different network parameters (e.g., AIFSN, CWmin, CWmax) to deal with the QoS required by the IoT applications. In the IEEE 802.11e standard, these parameters are defined by the demanded QoS. Although the default values assigned by the standard to these parameters are reasonable, in practice, when performance is measured by means of delays ($D$), packet loss ($L$), and throughput ($T$), the results are worse than when the network is left to operate without considering quality of service in the best-effort scenario, as in regular IEEE 802.11. Therefore, the network parameters should be tuned according to the network context to try to maximize the performance for that configuration.

Under this network model, the real-time controller (RT-controller) has two functions: to make SDWNs context-aware and to manage the QoS. The first one is to provide the Po-Fi Access Point the forwarding rules/tables for the messages based on the different priorities of them. The second function is to set the network configuration parameters mentioned before. To do this, it asks the intelligent agent for a new set of parameters that maximize the network performance. When the network configuration is changed by the arrival/departure of nodes or changes in the message priorities, the Po-Fi Access Point should demand the CAP controller for an update. If the QoS of the new message is already incorporated in the forwarding rules/table, there is only an UpdateRequest for the network parameters. Instead, if the message has a new QoS demand not already incorporated into the forwarding rules/table, the Po-Fi Access Point sends a PacketIn demand together with the UpdateRequest.

### 3.4. Intelligent Agent Description

The integration of an IA within the CAP controller aims to identify optimal network parameter configurations. It allows the CA-SDWN to optimize its performance in the current operation context. To do that, the agent performs an evolving process that considers two steps: (1) to predict the performance of a network in a particular configuration, and (2) to determine the best network configuration, in terms of performance, for that

context (Figure 6). After performing a configurable number of cycles, the IA obtains the best configuration for maximizing the network performance considering the context information recorded in its database.

In the first step, the agent employs a Random Forest regressor to predict network performance metrics $(D_i, T_i, L_i)$ based on a surrogate model trained on 150,000 simulation instances. Then, the Bayesian optimizer iteratively queries the predictive surrogate model to explore the configuration space and then determines the best parameter configuration considering the QoS and current demand of the network.

The integration between the performance predictor and the performance optimizer is represented in Figure 6. In particular, the optimizer iteratively updates CW parameters based on feedback received from the predictor. Then, in every cycle, it tries to minimize the objective score that considers the packet loss and delay. When the network performance becomes stable for a particular QoS, the agent provides the CW parameters to the real-time CAP controller.

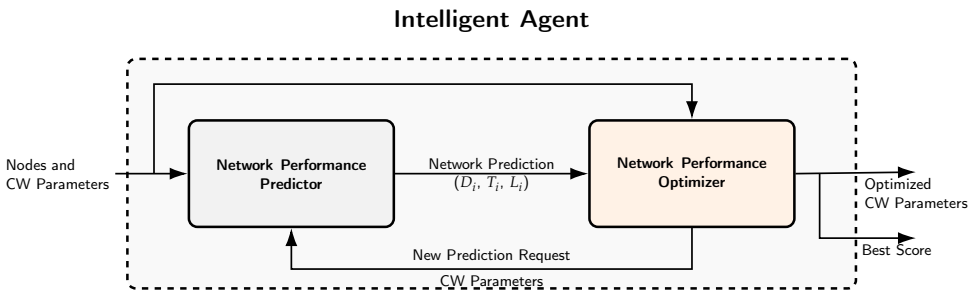



**Figure 6.** Interaction between the *network performance predictor* and the *Bayesian optimizer*.

### 3.4.1. Network Performance Predictor

The network performance predictor uses Random Forest [50,51] to make the predictions due to several reasons. First, it assembles methods that combine multiple decision trees, reducing variance and mitigating overfitting, an essential property when modeling complex stochastic systems such as wireless networks. Second, it handles mixed feature scales and non-linear interactions efficiently without requiring normalization or feature transformations. Finally, it offers interpretability through metrics that allow identifying the contention parameters influencing the network performance. These advantages make the Random Forest particularly suitable for performance prediction tasks based on simulation-derived datasets.

Figure 7 shows the block diagram of the network performance prediction process carried out by the intelligent agent. In this case, the model was optimized via RandomizedSearchCV across 300 iterations, resulting in a final architecture of 400 estimators with a maximum depth of 20, using a fixed random seed of 42 for reproducibility.

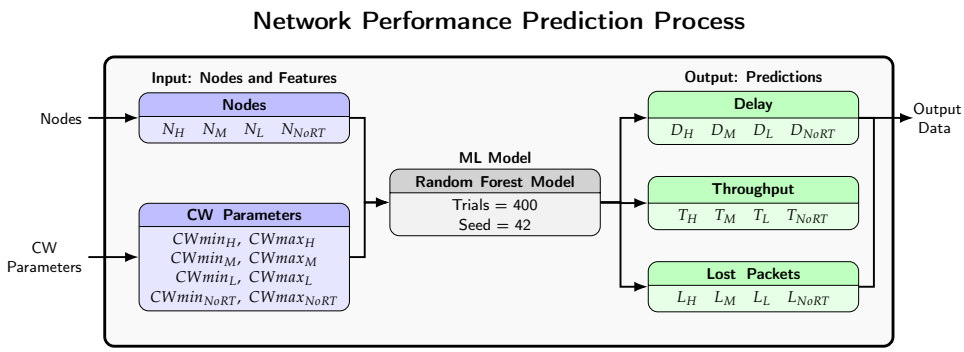



**Figure 7.** Process to predict the network performance for a certain configuration (network context).

The model effectively captures the non-linear dependencies between contention parameters and network behavior, serving as a reliable approximation of the system's performance under various configurations. Once trained, this model serves as a surrogate for the subsequent network performance optimization phase. The input features that feed into the Random Forest are the number of nodes $(N_i)$, $CWmin_i$ and $CWmax_i$ for each priority level $i \in \{H, M, L, NoRT\}$. To ensure data robustness, the training pipeline incorporates Winsorization to handle outliers and PowerTransformer (Yeo–Johnson) for statistical normalization. The algorithm learns to predict multi-target performance outcomes, such as delay, throughput, and packet loss $(D_i, T_i, L_i)$ across traffic classes.

By capturing the non-linear dependencies between contention parameters and network behavior, the model achieves high accuracy (validated via 5-fold cross-validation). Once trained on the full dataset, this model serves as a high-fidelity surrogate for the subsequent network performance optimization phase, providing a computationally efficient approximation of the system's behavior.

#### 3.4.2. The Bayesian Optimization Algorithm

In this stage, a *Bayesian optimization* algorithm [52,53], implemented using the *Optuna* framework, was employed to explore the configuration space and identify optimal contention parameters. The optimizer employed the *Tree-structured Parzen Estimator* (TPE) approach [54,55], which iteratively refines a probabilistic surrogate model of the objective function based on historical evaluations. The objective function was defined as $\min \sum w_i \cdot (L_i + D_i)$, where $L_i$ and $D_i$ correspond to packet loss and delay, respectively.

The Bayesian optimization method was selected due to its efficiency in optimizing expensive or black-box objective functions, typical in network systems, where each evaluation corresponds to a time-consuming simulation. Unlike exhaustive grid or random search methods, Bayesian optimization constructs a probabilistic surrogate of the objective function and balances the exploration of new regions with the exploitation of known good configurations. This enables a guided and computationally efficient search for optimal parameter values, even in high-dimensional spaces. The TPE variant, in particular, models the likelihood of achieving lower objective scores, dynamically adjusting its sampling strategy to favor configurations that are statistically more promising.

As shown in Figure 6, the optimization process operates in a closed-loop manner. Initially, the values assumed by the parameters are those proposed for the 802.11e standard. Subsequently, candidate sets of contention parameters $(CWmin_i, CWmax_i)$ are proposed by the Bayesian optimizer to the Random Forest predictor. Then, the latter determines the performance metrics $(L_i, D_i, T_i)$ for that configuration. These predictions are returned to the optimizer to compute a global performance score of the network, according to a custom objective function; in this case, it maximizes the performance.

In each new iteration, the feedback loop continues refining the optimizer's internal probability distributions, trying to identify a configuration that minimizes the objective score. This iterative process accelerates convergence by focusing the search on the most promising regions of the parameter space.

The optimization process relies on a custom scoring function that evaluates each candidate configuration, considering packet loss and delay across traffic classes. The function was designed to prioritize the performance of high- and medium-priority traffic, which typically corresponds to latency- and loss-sensitive applications. Low-priority traffic is also considered, although with slightly lower influence. Non-real-time (NoRT) traffic is not explicitly included in the objective function due to its lower performance sensitivity and higher tolerance to delay and packet loss. Accordingly, the scoring function is defined as:

$$\text{Score} = 0.35(L_H + D_H) + 0.35(L_M + D_M) + 0.30(L_L + D_L)$$

where $L_i$ and $D_i$ represent the averageof packet loss and delay for each traffic class. Since the objective of the Bayesian optimizer is to minimize this function, it inherently favors configurations that reduce packet loss and delay in high- and medium-priority traffic, while still maintaining acceptable performance levels for low-priority flows.

The coefficients used in the scoring function (0.35, 0.35, and 0.30) act as weighting factors that encode the QoS policy of the system. Because delay and packet loss values are normalized within the objective function, these weights do not compensate for scale differences between metrics; instead, they define the relative importance of each traffic class. A slightly higher emphasis is placed on high- and medium-priority traffic, whereas low-priority traffic has a marginally smaller influence on the final score. By excluding NoRT traffic from the objective function, the optimizer is allowed to focus computational effort on protecting the most critical services without over-constraining the search process.

In summary, this methodology combines large-scale network simulations, supervised learning, and Bayesian optimization to identify optimal contention parameter values. The integration of the Random Forest predictive model (Figure 7) with the TPE-based Bayesian search enables an intelligent and iterative exploration of the parameter space. This allows us to achieve an effective balance between accuracy, efficiency, and fairness among traffic priorities.

#### 3.4.3. Dataset to Train the IA

To identify a suitable set of contention parameters, 1000 different network configurations were generated. Each configuration was executed with five independent repetitions across ten different network sizes (from 10 to 100 devices), resulting in a total of 150,000 simulations. Each simulation lasted five minutes and was performed using the *modified SDWN*, varying the contention parameters (*CWmin*, *CWmax*) and the station distribution across the different traffic priorities (high, medium, low, and non-real-time). From the resulting simulation data, performance metrics such as *Throughput (Kbps)*, *Delay (ms)*, and *Packet Loss (%)* were extracted and aggregated by traffic priority, producing a comprehensive dataset that characterized the network behavior under multiple configurations.

This dataset was then used to train the Random Forest algorithm using the input and output variables summarized in Table 4. Specifically, the input features corresponded to the number of nodes and contention parameters ($N_i, CWmin_i, CWmax_i$) for each traffic priority level $i \in \{H, M, L, NoRT\}$, while the outputs included the performance metrics ($T_i, D_i, L_i$) for the same categories.

**Table 4.** Input and output parameters grouped by traffic priority.

| **Inputs** | | | | |
|---|---|---|---|---|
| **Parameter** | **Priorities** | | | |
| | **High** | **Medium** | **Low** | **NoRT** |
| Nodes | $N_H$ | $N_M$ | $N_L$ | $N_{NoRT}$ |
| CWmin | $CWmin_H$ | $CWmin_M$ | $CWmin_L$ | $CWmin_{NoRT}$ |
| CWmax | $CWmax_H$ | $CWmax_M$ | $CWmax_L$ | $CWmax_{NoRT}$ |
| **Outputs** | | | | |
| **Parameter (mean and std)** | **Priorities** | | | |
| | **High** | **Medium** | **Low** | **NoRT** |
| Lost Packets | $L_H$ | $L_M$ | $L_L$ | $L_{NoRT}$ |
| Delay | $D_H$ | $D_M$ | $D_L$ | $D_{NoRT}$ |
| Throughput | $T_H$ | $T_M$ | $T_L$ | $T_{NoRT}$ |

#### 3.4.4. Dynamic Network Parameter Update

The configuration recommendations of the intelligent agent are dynamically applied to the network through an event-driven update mechanism. This process ensures that EDCA parameters remain optimally tuned to the current network composition.

The update cycle is initiated by changes in network topology, specifically when a station connects to or disconnects from the Po-Fi Access Point. Upon detecting such an event, the access point updates its internal count of stations per priority class and sends an *UpdateRequest* message to the CAP controller (Figure 8). This message contains the current distribution of stations across the four priority categories; i.e., high ($N_H$), medium ($N_M$), low ($N_L$), and non-real-time ($N_{NoRT}$).

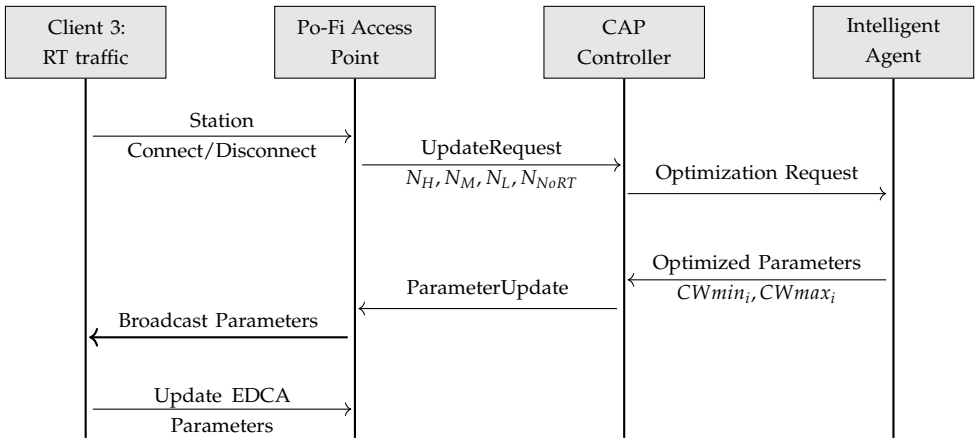



**Figure 8.** Sequence diagram for dynamic EDCA parameter updates based on network topology changes.

The CAP controller forwards this station distribution to the intelligent agent, which uses its pre-trained model to determine the optimal $CWmin$ and $CWmax$ values for each priority level, given the current network size and composition. This agent returns the optimized parameters to the CAP controller, which then propagates them to the Po-Fi Access Point via a *ParameterUpdate* message.

Finally, the Po-Fi Access Point applies the new EDCA parameters to its transmission queues. Each network node subsequently updates its contention parameters based on its traffic priority: high-priority stations apply the $(CWmin_H, CWmax_H)$ values, medium-priority stations use $(CWmin_M, CWmax_M)$, and so on. This ensures that all network devices operate with contention parameters optimized for the current network context.

## 4. Evaluating the Performance of CA-SDWNs

To analyze the behavior and performance of the networks operating with the controller under different configurations and traffic patterns, several experiments were carried out as summarized in Table 5. These experiments were designed to evaluate the impact of QoS management mechanisms, EDCA parameter adjustments, and different combinations of traffic categories and packet sizes on key performance metrics.

**Table 5.** Summary of the experimental evaluation with the different mechanisms to control the EDCA parameters.

| Mechanism | CA-SDWN | QoS | Interval(s) | Packet Size (Bytes) | Categories | EDCA Modified | Method |
|---|---|---|---|---|---|---|---|
| 1 (Best Effort) | No | No | 1 s | 256, 512, 1024 | High+ Medium+ Low+ Non-Real-Time | No | |
| 2 (Orig. SDWN) | Yes | Yes | 1 s | 256, 512, 1024 | High+ Medium+ Low+ Non-Real-Time | No | |
| 3 (Mod. SDWN) | Yes | Yes | 1 s | 256, 512, 1024 | High+ Medium+ Low+ Non-Real-Time | Yes | Fixed |
| 4 (IA SDWN) | Yes | Yes | 1 s | 256, 512, 1024 | High+ Medium+ Low+ Non-Real-Time | Yes | Based on ML |

The baseline mechanism utilized for QoS management was the 802.11 best-effort, i.e., the regular proposal of the standard that lacks explicit QoS prioritization or control logic. In this case, the channel contention was resolved using the standard IEEE 802.11 binary exponential backoff mechanism.

The second mechanism involved the use of the controller with the default EDCA parameters for 802.11e. In this approach (labeled as *Orig. SDWN* in Table 5), the contention windows were shorter than the ones used for low-priority traffic. High-priority hosts use these windows before retransmitting after a detected collision.

In the third mechanism (labeled as *Mod. SDWN*), these parameters were changed for a set of fixed ones determined empirically, but we kept the AIFSN value defined by the standard and modified the values for CWmin and CWmax. In order to determine the optimal tuple (AIFSN, CWmin, CWmax) we followed the method proposed in [56] and chose the combination that caused the fewest variations to the standard.

Finally, in the fourth mechanism (labeled as *IA SDWN*) the network used the proposed intelligent agent, which determined the best set of parameters for each case. The Po-Fi Access Point and the CAP controller utilized in the previous configuration were held, but the contention parameters were modified. Moreover, different traffic priorities and CWmin–CWmax pairs were used; particularly, CWmin and CWmax values were those provided by the intelligent agent.

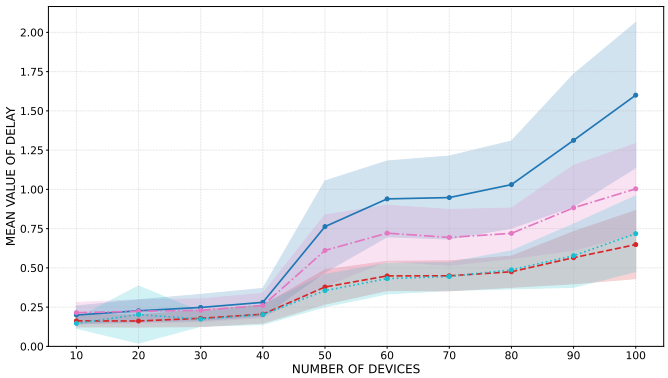
The experiments involved mainly fixed nodes, as is typical in sensor wireless networks applied in business domains like precision agriculture, Industry 4.0/5.0, disaster relief and smart environments. In these scenarios, mobile nodes usually represent a small percentage of the total; typically, they are used to implement autonomous vehicles or support mobile workers' activities.

In the simulations, the deployment of nodes was done randomly with the following guidelines. Eighty percent of the nodes were stationary, positioned within a 0.6 transmission radius of the AP. In order to do that, we used the NS-3 *UniformDiscPositionAllocator* model. The remaining nodes (i.e., 20% of them) were mobile and used the NS-3 *RandomWalk2d* mobility pattern. These nodes moved within the transmission range of the AP.

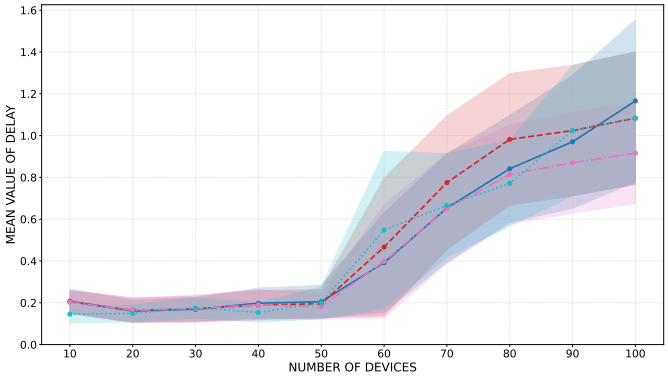
For each simulation scenario, the performance was evaluated using three different packet sizes (256, 512, and 1024 bytes), and the traffic was distributed across the four priority levels. For each combination of packet size and network configuration, ten different networks were randomly generated with 10 to 100 hosts each, in steps of 10 hosts. The generation periods for sending high, medium, and low/background messages were set to 1 s for all simulations. These experiments included 1200 random networks (four scenarios, three different packets' sizes, 10 different networks' sizes, 10 instances of each one), in each case simulated for ten minutes.

### 4.1. Experimental Results

The dataset with the evaluation results is publicly available at [21]. The results showed that the relative behavior of the four methods was similar for the different QoS and packets sizes. Therefore, for the detailed analysis of results, we selected just a few combinations of these variables. Figures 9 and 10 present the average delay obtained for the different network sizes for the 256- and 1024-byte packets sizes for high and low priorities. Similarly, Figures 11 and 12 show the values of the packet loss for the same network configurations. For all cases, the 95% confidence interval, computed with the t-student procedure [57], is represented as a colored region around each line.

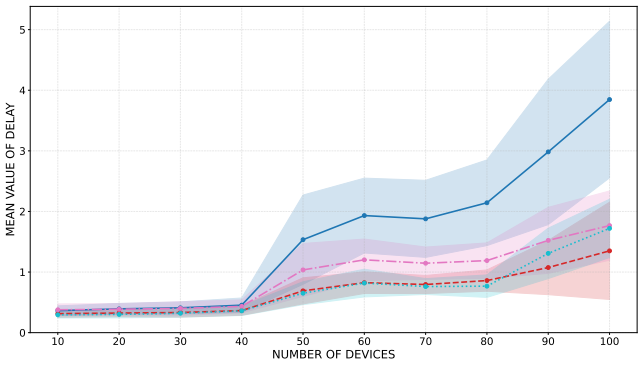



(**a**) High Priority

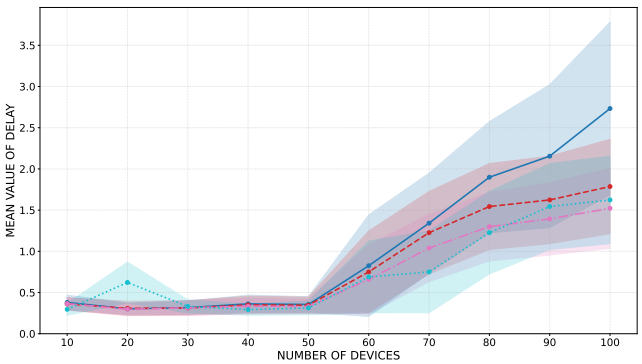



(**b**) Low Priority

**Figure 9.** Transmission delay for 256 B packet size. In blue are the results for the *best-effort* mechanism, in red *Orig. SDWN*, in pink *Modif. SDWN* and in light blue *IA SDWN*.

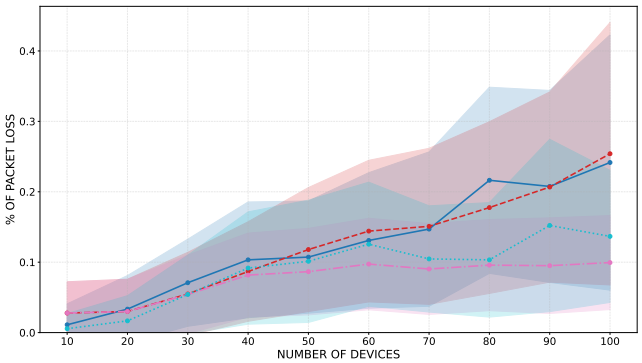



(**a**) High Priority

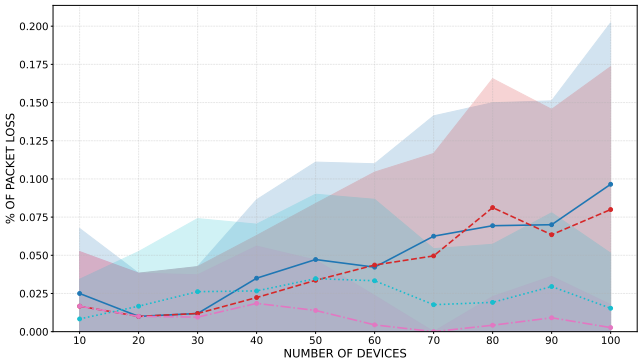



(**b**) Low Priority

**Figure 10.** Transmission delay for 1024 B packet size. In blue the *best effort*, in red *Orig. SDWN*, in pink *Modif. SDWN* and in light blue the *IA SDWN*.

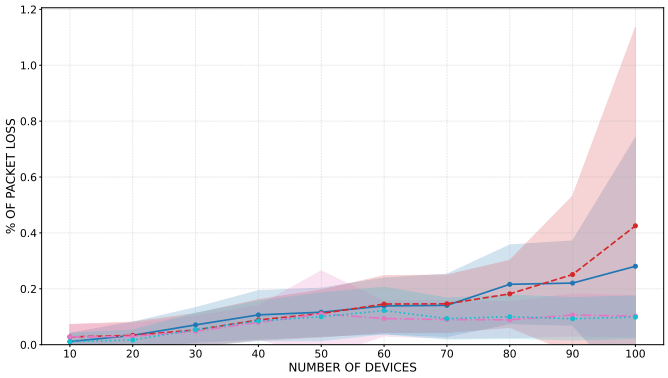



(**a**) High Priority

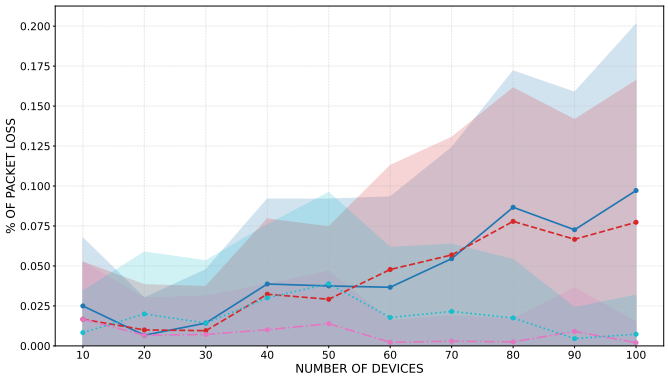



(**b**) Low Priority

**Figure 11.** Percentage of packet loss for 256B packet size. In blue the *best effort*, in red *Orig. SDWN*, in pink *Modif. SDWN* and in light blue the *IA SDWN*.




(**a**) High Priority




(**b**) Low Priority

**Figure 12.** Percentage of packet loss for 1024B packet size. In blue the *best effort*, in red *Orig. SDWN*, in pink *Modif. SDWN* and in light blue the *IA SDWN*.

The results for the rest of the configurations show negligible differences with those of the previous figures. In particular, the results for medium-priority traffic are between those of high and low traffic, and for non-real-time traffic, they are below the low-priority case. When using packets of 512 bytes, the results are between those shown in the previous figures. Next, we analyze in more detail the transmission delay and the packet loss.

#### 4.1.1. Analysis of the Transmission Delay

The transmission delay considers the time needed to send a packet from the source to the destination node, and it is affected by the time window sizes each host has to wait before trying to transmit again, i.e., the parameters CWmin and CWmax.

Figures 9 and 10 present the delay for high- and low-priority traffic, when schedule with the four mechanisms and considering packets sizes of 256 and 1024 bytes respectively.

The figure shows similar performance for the four mechanisms (i.e., delays around 0.5 ms or less) up to network sizes of 40 nodes, regardless of the traffic priority. In larger networks the performance decrease for all the mechanisms. The worst case is always the *best-effort* method. For high-priority traffic and networks sizes of 40–80 nodes, the *IA SDWN* and *Orig. SDWN* mechanisms have similar performance. However, for networks with 90 and 100 nodes, the *Orig. SDWN* has the best performance. This is because it has the shortest backoff windows, which produce the shortest response in case of collisions. In the low-priority traffic, the *Orig. SDWN* is not the best option, and both the *Modif. SDWN* and *IA SDWN* perform better.

#### 4.1.2. Analysis of the Packet Loss

A packet loss occurs when a packet is ultimately discarded by the host or access point after multiple failed retransmission attempts. In collision conditions, the node executes an exponential backoff algorithm based on contention windows until the specified attempt limit is reached. Therefore, the percentage of packet loss directly influences the network performance.

In case of these experiments, the results show a very low percentage of packet loss in all simulated scenarios (below 1%). It means that this aspect does not represent a problem in networks that involve up to 100 nodes, e.g., networks that support IoT systems to perform autonomous irrigation in farms. However, it could represent an issue in larger IoT infrastructures, like in volcanoes' monitoring systems or tsunamis' early warning systems, where thousands of nodes are deployed in the field.

If we consider the tendencies shown in Figures 11 and 12, we can see that the *best-effort* mechanisms had the same performance independently of the message's priority. In the case of mechanisms that use controllers, *Orig. SDWN* had the worst performance. This is due to the fact that CWmin and CWmax are the smaller among the alternatives that use controllers. This produces in some way a denial-of-service effect because, while attempting to transmit immediately after a collision is detected, the probability of producing a new collision is increased. In the cases of *Modif. SDWN* and *IA SDWN* a very similar performance is shown for both mechanisms, but the former outperforms the latter because the used backoff windows are larger.

Unlike the fixed and overly aggressive values of the standard mechanism, the *Modif. SDWN* and *IA SDWN* employ optimized and larger contention windows. This increases the backoff range, effectively spacing out transmission attempts and providing a larger margin for retransmissions before a packet is discarded, thereby reducing collisions. Furthermore, by retaining baseline AIFSN values, traffic class differentiation is preserved, ensuring that high-priority flows maintain preferential channel access during contention. The network not only achieves high reliability but also exhibits good operational resilience and fast recovery capability during congestion episodes, outperforming the other methods.

### 4.2. Analysis of the CA-SDWN Adaptability

In terms of network performance, the previous results showed no significant differences among the four analyzed approaches, considering every configuration. However, the *IA SDWN* is the only mechanism that allows the network (via the CA-Controller) to self-adapt its behavior to changes in the context, i.e., to make the network context-aware.

In order to illustrate that capability, we performed a second round of experiments to evaluate the adaptability of the IA-based approach considering changes, first in the objective function and then in the network traffic. As explained in Section 3.4.2, the objective function can be tuned on the fly to reduce the delay or packet loss according to the needs of the system being supported.

The simulated scenario involved 100 nodes and a packet size of 1024 bytes, where the network behavior was analyzed under different conditions. Contrary to the previous experiments, which considered equal weight for the delay and packet loss, in this case we modified these weights to optimize first the delay, and then the packet loss ratio.

In the first case, the reduction in the delay was 30% (from 1.7 ms to 1.2 ms), and that value became the lowest among all the methods evaluated. Then, when the objective function was modified to improve the packet loss ratio, we found no significant changes in the values of that variable. This suggests that the optimization process had already converged to a global minimum for packet loss.

After changing the delay and packet loss, we performed a new round of simulations by keeping the objective function as those reported in Section 4.1, i.e., using equal weights for delay and packet loss. However, in this case we increased the proportion of high-priority traffic from 25% to 40%. In this scenario, the number of lost packets was reduced by approximately 50% (from 0.099 to 0.051). This was the lowest value among all the methods used and in particular was 86% less than the value reached by the *Orig. SDWN* mechanism.

Another simulation was performed with 40% high-priority traffic but with the objective function optimizing the delay. In this case, the *IA SDWN* improved the values, becoming the method with the lowest delay and also with similar packet loss to the other methods. When the optimization was for the packet loss, the results showed minimum values in that variable, but results were not so good in terms of delay.

This experiment shows that using the *IA SDWN* mechanism, the CA-Controller can self-adapt its behavior and provide better performance when there are changes in the network configuration, in the type of traffic or in the objective function.

## 5. Discussion

The results presented in Section 4 reveal several key insights regarding the application of IEEE 802.11e and SDWN-based QoS strategies in wireless networks; specifically, in scenarios where traffic prioritization is required beyond traditional multimedia use cases.

First, the introduction of a CAP controller enables the dynamic management of EDCA parameters, adapting them to the current network size, composition and traffic. The evaluation results show that only when EDCA is modified with optimized contention window values does the combination of QoS and CA-SDWN provide an improvement over both best-effort and standard EDCA mechanisms. This improvement is achieved by reducing collision rates, allowing for more efficient retransmission attempts, while preserving prioritization through AIFSN differentiation. The outcome is a low packet loss rate, consistently maintained at or near 0%, even in high-density scenarios. This is a performance level that is not marginal but fundamental for reliable operation in dense networks.

Second, the number of hosts is a major factor that influences network performance. Although IEEE 802.11 standards allow up to 255 clients per access point, both simulations and practical experience suggest that network performance degrades beyond 40 devices

under conventional configurations. In our simulations, the standard EDCA suffered performance degradation after that threshold, with high-priority traffic being paradoxically penalized. However, the proposed *Modif. SDWN* and *IA SDWN* mechanisms, successfully counteracted this trend, maintaining near-zero packet loss even with 100 devices, as shown in Figures 11 and 12. These results not only underscore the importance of proper network engineering but also demonstrate that parameter optimization can effectively raise the practical device limit per access point.

Third, the use of the *IA SDWN* mechanism allows the network to consider the current context and self-adjust its behavior in case of changes in the network size, composition or objective function. Since these aspects can be detected, they can be considered by adjusting the network parameters on the fly. This capability makes the CA-SDWN not only flexible in terms of behavior but also context-aware. IoT systems that involve a dynamic context, like those supporting search-and-rescue operations, massive evacuations, or simply involving a high percentage of nomad users, can take advantage of this capability. Therefore, the CA-SDWN represents a step forward in managing QoS in SDWNs.

## 6. Conclusions and Future Work

This paper presented CA-SDWN, a context-aware architecture for Software-Defined Wireless Networks designed to manage quality of service (QoS) in dynamic IoT environments. By integrating a machine learning-based intelligent agent with a Bayesian optimization framework, the proposed solution addresses the inherent challenges of traditional IEEE 802.11e EDCA mechanisms, which often fail to adapt to fluctuating network densities and traffic demands.

Moreover, the CA-SDWN can modify its behavior on the fly by adjusting the operation parameters. This capability turns these software infrastructures into context-aware networks, particularly useful to manage QoS in operating scenarios that involve dynamic network contexts.

The experimental results, conducted via NS-3 simulations, indicated that CA-SDWNs tended to outperform the other analyzed network proposals, i.e., the standard IEEE 802.11 (best-effort), 802.11e (original) and modified SDWN, considering the overall results in terms of delay and packet loss for several traffic priorities. In addition, the CA-SDWN approach was the sole proposal able to self-adapt its behavior on the fly to manage traffic changes.

These results show that using conventional WiFi technologies to implement SDWNs, when enhanced with adaptive intelligence, is a cost-effective and reliable alternative to support real-time IoT applications without requiring 5G or dedicated wired infrastructures. To allow reproducibility of the experiments and improving this proposal by other researchers, the full implementation of the CA-SDWN as well as the new NS-3 components have been made publicly available in a GitHub repository [20].

Future work in this research initiative includes deploying the CA-SDWN architecture on open-hardware access points to validate its performance under real traffic conditions, measuring the responsiveness of the closed-loop control mechanism. Additionally, we will explore multi-objective optimization within the intelligent agent to simultaneously consider delay, packet loss, throughput, and energy consumption, critical factors for large-scale, battery-powered IoT deployments.

**Author Contributions:** Conceptualization, R.M.S.; methodology R.M.S. and D.G.R.; software D.G.R.; validation R.M.S., D.G.R. and S.F.O.; formal analysis R.M.S., D.G.R. and S.F.O.; investigation D.G.R. and R.M.S.; resources S.F.O. and R.M.S.; data curation D.G.R.; writing D.G.R., R.M.S. and S.F.O.; draft preparation R.M.S., S.F.O. and D.G.R.; writing—review and editing S.F.O., R.M.S. and D.G.R.; visualization D.G.R., S.F.O. and R.M.S.; supervision R.M.S. and S.F.O.; project administration R.M.S.

and S.F.O.; funding acquisition S.F.O. and R.M.S. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research was funded by CONICET grant number 11220220100066CO.

**Data Availability Statement:** Source code: https://github.com/dainiergonzalezromero/WiFi-QoS-NS3.git (accessed on 15 March 2026); Data set: https://data.mendeley.com/datasets/ptjvmhb5dc/1 (accessed on 15 March 2026).

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Appendix A. Defining CA-SDWNs in the NS-3 Simulator

This appendix presents the implementation and configuration of a CA-SDWN in NS-3, in order to evaluate its performance under different simulation scenarios. The simulator is a well-known open-source software platform that allows for simulating a wide range of networks, including wireless, mobile and sensor networks. These simulations address the whole network stack, from the physical layer to the application layer. The simulator allows the capture and analysis of network metrics, such as transmission rate, latency, and packet loss, making it a valuable tool for evaluating network performance under several conditions.

### *Appendix A.1. Simulation of a CA-SDWN*

The simulations considered a network environment where a central Po-Fi Access Point distributed connectivity to multiple devices (e.g., sensors and actuators). By modulating parameters such as the TXOP limit and access category (AC) assignments, real-time traffic was prioritized over lower-priority flows. This approach facilitated efficient resource management while optimizing throughput and minimizing latency for time-critical applications. The next subsections describe the stages considered in the simulation process.

### *Appendix A.2. Network Configuration for Using the NS-3 Modules*

Implementing the network required first configuring the network structure and the mechanisms implemented to ensure QoS. Then, the simulations used the NS-3 modules and the previous configurations to create a simulated SDWN and analyze its performance.

For this purpose, the NS-3 modules were integrated following a logic inspired by the TCP/IP model, ensuring coherence in the interdependence between layers, as illustrated in Figure A1. The configuration proceeded in a bottom-up manner, starting from fundamental abstractions and building up to application-level services.

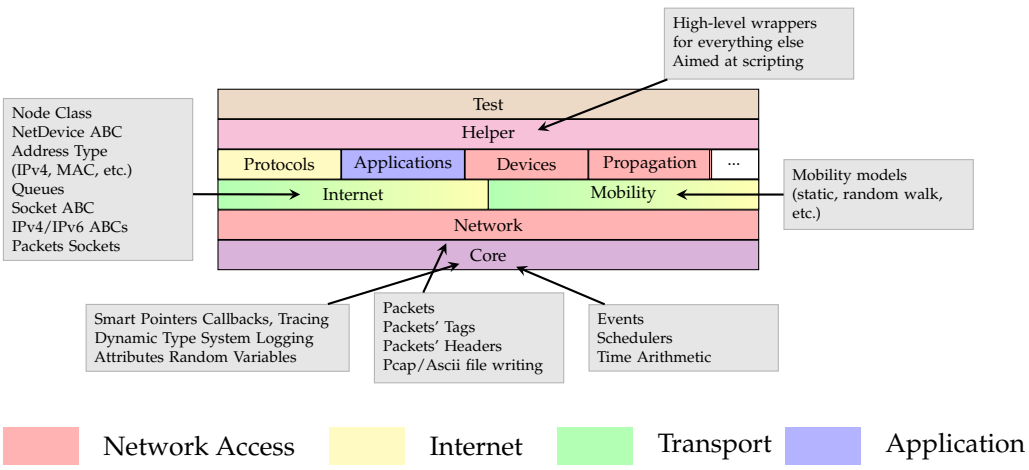



**Figure A1.** Software Organization of NS-3 colored according to the TCP/IP layers (adapted from [58]).

At the foundation lay the Core Module, which provided the discrete-event simulation engine. This module offered essential services such as event scheduling, time management, callbacks, tracing, and the attribute system. Although it did not directly correspond to any specific TCP/IP layer, it constituted the infrastructure upon which all other abstractions were built.

The Network Module was located over this (core) layer and introduced the central communication object, i.e., the *packet*. This module managed encapsulation, headers, trailers, and tags, supporting both the frame operations of the network access layer and the packet-level mechanisms of the *Network layer* in the TCP/IP stack.

As illustrated in Figure A1, each module is color-coded according to its correspondence with the TCP/IP model. In particular, red modules represent the *network access* layer, yellow the *Internet* layer, green the *Transport* layer, and blue the *Application* layer. Additional modules without a direct TCP/IP counterpart (e.g., Core, Mobility, Helper, Test) are highlighted in distinct colors to emphasize their supporting role within the NS-3 architecture.

The next level considers two key modules, `Internet` and `Mobility`, and both consider Internet and transport aspects. The first one implements the `Node` class, `NetDevice` interfaces, address types (MAC, IPv4, IPv6), queues, sockets, and protocol stacks such as IPv4, IPv6, TCP, and UDP. Within the TCP/IP model, this module covers the functionalities of the *Internet layer* (e.g., node addressing and routing) and the *Transport layer* (TCP/UDP). The second one, i.e., the `Mobility Module`, although lacking a direct counterpart in TCP/IP, is fundamental in NS-3 to define node positions and mobility patterns, which are particularly critical in wireless scenarios that consider mobile nodes.

The next level in NS-3 includes the following modules:

- `Protocols.` It includes supporting mechanisms such as ARP and dynamic routing, extending the *Internet layer*.
- `Applications.` It implements traffic generators and services, corresponding to the *Application layer*.
- `Devices.` It models network interfaces and channels, corresponding to the *Network Access layer*.
- `Propagation.` It provides wireless propagation models (path loss, shadowing, fading), also within the *Network Access layer*.

The next layer considers mainly the `Helper Module`, which simplifies the construction of simulation scenarios through supporting classes (e.g., `MobilityHelper`, `InternetStackHelper`, `Ipv4AddressHelper`). Although all functionalities can also be implemented via the low-level API, helpers significantly reduce code complexity by providing higher-level abstractions and bulk configurations.

Finally, the `Test Module` provides a comprehensive and systematic environment for the validation and verification of NS-3 models. Its purpose is to ensure that simulations consistently meet essential requirements, such as correctness, robustness, performance, maintainability, and reproducibility. The module supports multiple levels of testing, ranging from unit and integration tests to system-level evaluations. Moreover, it incorporates both verification, ensuring compliance with formal specifications, and validation, assessing the suitability of the models for their intended use. Although it was not directly applied in this work, the Test Module is a fundamental component of the NS-3 architecture, playing a critical role in ensuring the accuracy, reliability, and overall credibility of simulation results.

Overall, this modular organization of NS-3 enables not only the construction of network scenarios inspired by the TCP/IP architecture but also the integration of additional mechanisms (such as QoS and SDWN) based on queuing systems, scheduling algorithms, and dynamic routing policies. In this way, the simulation realistically captures both the hierarchical structure of the network and the impact of quality-of-service mechanisms on overall performance.

*Appendix A.3. Node Modeling and Distribution*

The NS-3 simulation environment was configured according to the IEEE 802.11n standard with support for IEEE 802.11e, enabling the implementation of QoS mechanisms. The network topology comprised a Po-Fi Access Point, a CAP controller, and a variable number of stations (i.e., end-devices). One node was designated as the Po-Fi Access Point, while the others functioned as stations.

The WiFi network operated on channel 36 in the 5 GHz band with a channel width of 20 MHz. Both the Access Point and the stations were configured with the same SSID to ensure proper association. The physical layer parameters were set to model realistic wireless channel behavior, and the MAC layer was configured to comply with the IEEE 802.11n standard, ensuring compatibility across devices.

To implement the support for QoS on an IEEE 802.11e network, the access point was set to operate with QoS enabled, while stations were assigned one of four access categories: *non-real-time* (BK: background), *real-time lowest priority* (BE: best-effort), *real-time medium priority* (VI: video), and *real-time high priority* (VO: voice). This configuration type ensured the QoS assignment.

QoS parameters of each station were individually configured by cycling through the access categories and applying the corresponding EDCA settings. The Po-Fi Access Point was further tuned by enabling QoS support, specifying the SSID, defining the maximum size of aggregated MAC Protocol Data Units for the selected category, and disabling active probing to reduce unnecessary management traffic.

Once the device was installed, the corresponding QoSTxop object was accessed, and the parameters AIFSN, CWmin, and CWmax were configured according to the EDCA profile. This approach allowed for fine-grained, per-station MAC parameter configuration, providing better adaptation to different traffic types, and enhancing service differentiation as defined in IEEE 802.11e.

For spatial distribution, the Po-Fi Access Point was placed in the center of the network area, and the stations were randomly located within a radius of 50 m. Node mobility was not considered in these scenarios, and the stations remained stationary throughout the simulations.

The network was configured with the TCP/IP stack, assigning IP addresses within the 192.168.1.0/24 subnetwork. Each station generated UDP traffic directed to the access point, using variable packet sizes of 256, 512, and 1024 bytes. The same experiments were conducted for each of these packet sizes in order to analyze the effect that their variability produced on network performance.

Finally, the QoS support was reinforced by the use of the Type of Service (ToS) field, located in the IP header, which was mapped to the TiD in NS-3. This approach, dynamically managed by the CAP controller, classified the traffic into four priority categories, allowing us to optimize resource utilization and ensuring differentiated treatment according to the service type and network context.

*Appendix A.4. Implementation of Device Registration and Packet Management*

Figure A2 shows the registration and packet management process for real-time and non-real-time clients. The process operates on a SDN-based WiFi architecture with centralized flow control that uses a CAP controller and a Po-Fi Access Point. The controller classifies flows by TiD values and determines transmission limits, while the access point manages packet reception, queuing, and prioritized forwarding with dynamic Txop configurations.

During initialization, the UDP port is configured for communication, the socket is initialized, signals are linked to capture incoming IPv4 events, and a callback function is set to handle incoming packets. Then, when a client device (e.g., a sensor or actuator) sends a packet (action shown as (1) in Figure A2), it is received by the access point (2).

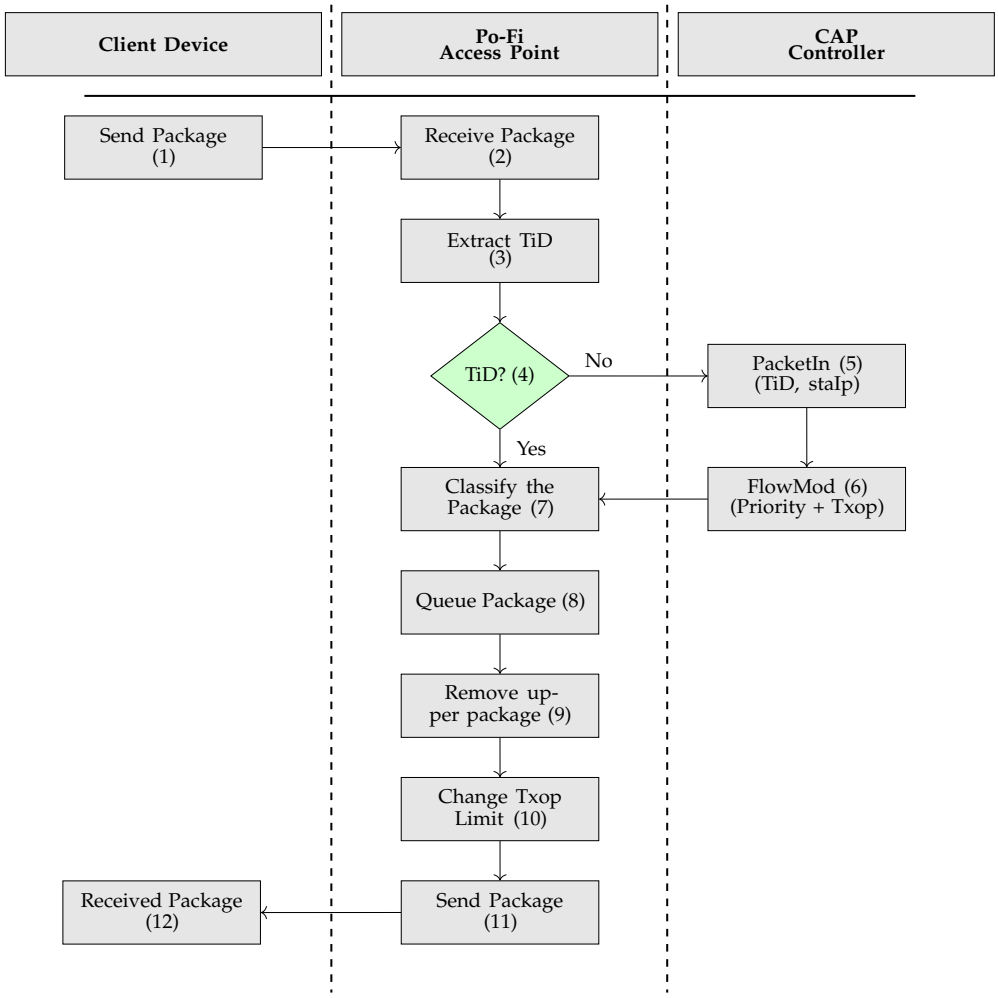



**Figure A2.** Optimized registration and packet management process for real-time and non-real-time clients in a Po-Fi Access Point.

Upon receiving data through its socket, the access point extracts the ToS (3) field from the packets, which corresponds to the TiD, and updates the TiDMap to associate IP addresses with their respective TiD values. Then, it processes each packet by querying the tidRegistry map to check if a FlowMod rule already exists for the received TiD (4). If such a rule does exist, a new query is sent to the CAP controller via `PacketIn` (5).

The CAP controller acts as the logical core of the system, which is in charge of making decisions about the priority and transmission limit of the flows. When it receives a `PacketIn` message (5), it obtains the TiD field and the IP address of the sender node. Then, it returns a `FlowMod` structure containing an assigned priority category (high, medium, low) and a Txop limit in microseconds (6). This assignment follows static rules based on the TiD value. This method is essential for flow classification and medium access control at the MAC layer.

Once the `FlowMod` message is received by the Po-Fi Access Point, the packet is classified according to the priority and Txop assigned (7) to its TiD and placed in the corresponding priority queue (8). The packet at the front of the highest-priority queue is then selected and removed (9). Before transmission, the Po-Fi Access Point adjusts the Txop limit on its WiFi interface (10) by configuring the QoSTxop object of the relevant priority level (high, medium, low, and non-real-time), ensuring efficient and prioritized medium access. The packet is then transmitted to the recipient (11).

Performance metrics, such as latency and jitter, were calculated using arrival and dispatch times and stored for later analysis. Finally, these metrics, along with received and sent

packets, bytes, and throughput in Kbps, were collected and exported to a CSV file, which was essential for evaluating the effectiveness of the applied QoS management approach.

In addition, we implemented other actions that enabled a comprehensive analysis and validation of the system. These actions included monitoring network flows that allowed the collection of key metrics (e.g., throughput, latency, and packet loss), as well as flow classification.

To facilitate visualization of network behavior, animations were generated in NetAnim, using labeled nodes and real-time statistics. In addition, traces were generated in ASCII and PCAP formats, allowing detailed traffic analysis using tools like Wireshark. These complementary configurations not only enriched the simulation but also provided essential tools for debugging, optimizing, and evaluating the effectiveness of the QoS policies implemented in the SDWN.

## References


1. Santos, R.M.; Santos, J.; Orozco, J.D. A least upper bound on the fault tolerance of real-time systems. *J. Syst. Softw.* **2005**, *78*, 47–55. [CrossRef]
2. Akyildiz, I.; Su, W.; Sankarasubramaniam, Y.; Cayirci, E. A survey on sensor networks. *IEEE Commun. Mag.* **2002**, *40*, 102–114. [CrossRef]
3. Balasubramanian, V.; Aloqaily, M.; Reisslein, M. An SDN architecture for time sensitive industrial IoT. *Comput. Netw.* **2021**, *186*, 107739. [CrossRef]
4. Rangisetti, A.K.; Tamma, B.R. Software Defined Wireless Networks: A Survey of Issues and Solutions. *Wirel. Pers. Commun.* **2017**, *97*, 6019–6053. [CrossRef]
5. Kumar, R.; U., V.; Tiwari, V. Optimized traffic engineering in Software Defined Wireless Network based IoT (SDWN-IoT): State-of-the-art, research opportunities and challenges. *Comput. Sci. Rev.* **2023**, *49*, 100572. [CrossRef]
6. Rahman, A.; Wadud, A.; Islam, J.; Kundu, D.; Bhuiyan, T.; Muhammad, G.; Ali, Z. Internet of medical things and blockchain-enabled patient-centric agent through SDN for remote patient monitoring in 5G network. *Sci. Rep.* **2024**, *14*, 5297. [CrossRef]
7. Benotmane, M.; Elhari, K.; Kabbaj, A. Survey of IoT Reference Architectures and Models and IoT Initiatives. In *Future Technologies Conference (FTC) 2021*; Arai, K., Ed.; Springer: Cham, Switzerland, 2022; Volume 1, pp. 294–320.
8. Saemaldahr, R.; Thapa, B.; Maikoo, K.; Fernandez, E.B. Reference Architectures for the IoT: A Survey. In *Innovative Systems for Intelligent Health Informatics*; Saeed, F., Mohammed, F., Al-Nahari, A., Eds.; Springer: Cham, Switzerland, 2021; pp. 635–646.
9. Vera, T.; Ochoa, S.F.; Santos, R.; Perovich, D. Defining the Scope of IoT Systems at the Pre-contract Time. In *International Conference on Ubiquitous Computing and Ambient Intelligence (UCAmI 2024)*; Bravo, J., Nugent, C., Cleland, I., Eds.; Springer: Cham, Switzerland, 2024; pp. 666–673.
10. ZTE Corporation. *WiFi 6 Technology and Evolution White Paper*; Technical Report; ZTE Corporation: Shenzhen, China, 2020.
11. Miuccio, L.; Panno, D.; Pisacane, P.; Riolo, S. A QoS-aware and channel-aware Radio Resource Management framework for multi-numerology systems. *Comput. Commun.* **2022**, *191*, 299–314. [CrossRef]
12. Alnaser, A.a.M.A.a.; Saloum, S.S.; Sharadqh, A.A.; Hatamleh, H. Optimizing multi-tier scheduling and secure routing in edge-assisted software-defined wireless sensor network environment using moving target defense and AI techniques. *Future Internet* **2024**, *16*, 386. [CrossRef]
13. Al-Hamid, D.Z.; Karegar, P.A.; Chong, P.H.J. A novel SDWSN-based testbed for IoT smart applications. *Future Internet* **2023**, *15*, 291. [CrossRef]
14. Alzahrani, B.; Fotiou, N. Securing sdn-based iot group communication. *Future Internet* **2021**, *13*, 207. [CrossRef]
15. Xu, C. Resource Optimization Algorithm for 5G Core Network Integrating NFV and SDN Technologies. *Int. J. Intell. Netw.* **2025**, *6*, 36–46. [CrossRef]
16. Narwaria, A.; Mazumdar, A.P. Software-Defined Wireless Sensor Network: A Comprehensive Survey. *J. Netw. Comput. Appl.* **2023**, *215*, 103636. [CrossRef]
17. Fogli, M.; Giannelli, C.; Stefanelli, C. Software-Defined Networking in wireless ad hoc scenarios: Objectives and control architectures. *J. Netw. Comput. Appl.* **2022**, *203*, 103387. [CrossRef]
18. Llinas, A.; Micheletto, M.; Santos, R.; Ochoa, S. Software defined wireless networks with real-time constraints. In *12th Latin-American Symposium on Dependable and Secure Computing*; LADC '23; Association for Computing Machinery: New York, NY, USA, 2023; pp. 226–229. [CrossRef]
19. IEEE. *IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems—Local and Metropolitan Area Networks–Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*; Technical Report; Institute of Electrical and Electronics Engineers: Piscataway, NJ, USA, 2021. [CrossRef]

20. González Romero, D. CA-SDWN-WiFi-QoS: AI-Driven Knowledge-Defined Wireless Networks (KDWN). 2026. Available online: https://github.com/dainiergonzalezromero/WiFi-QoS-NS3.git (accessed on 15 March 2026).
21. González Romero, D.; Santos, R.; Ochoa, S.F. Evaluation Results of Context-Aware Software Defined Wireless Networks. 2026. Available online: https://data.mendeley.com/datasets/ptjvmhb5dc/1 (accessed on 15 March 2026).
22. McKeown, N.; Anderson, T.; Balakrishnan, H.; Parulkar, G.; Peterson, L.; Rexford, J.; Shenker, S.; Turner, J. OpenFlow: Enabling innovation in campus networks. *SIGCOMM Comput. Commun. Rev.* **2008**, *38*, 69–74. [CrossRef]
23. Luo, T.; Tan, H.P.; Quek, T.Q.S. Sensor OpenFlow: Enabling Software-Defined Wireless Sensor Networks. *IEEE Commun. Lett.* **2012**, *16*, 1896–1899. [CrossRef]
24. Yan, M.; Casey, J.; Shome, P.; Sprintson, A.; Sutton, A. ÆtherFlow: Principled Wireless Support in SDN. In *2015 IEEE 23rd International Conference on Network Protocols (ICNP)*; IEEE: Piscataway, NJ, USA, 2015; pp. 432–437.
25. Guan, Z.; Bertizzolo, L.; Demirors, E.; Melodia, T. WNOS: Enabling Principled Software-Defined Wireless Networking. *IEEE/ACM Trans. Netw.* **2021**, *29*, 1391–1407. [CrossRef]
26. Shi, Z.; Tian, Y.; Wang, X.; Pan, J.; Zhang, X. Po-Fi: Facilitating innovations on WiFi networks with an SDN approach. *Comput. Netw.* **2021**, *187*, 107781. [CrossRef]
27. Theodorou, T.; Mamatas, L. DENIS-SDN: Software-Defined Network Slicing Solution for Dense and Ultra-Dense IoT Networks. *arXiv* **2023**, arXiv:2312.13662.
28. Galluccio, L.; Milardo, S.; Morabito, G.; Palazzo, S. SDN-WISE: Design, prototyping and experimentation of a stateful SDN solution for WIreless Sensor networks. In *2015 IEEE Conference on Computer Communications*; IEEE: Piscataway, NJ, USA, 2015; pp. 513–521. [CrossRef]
29. Alves, R.C.A.; Oliveira, D.A.G.d.; Núñez Segura, G.A.; Margi, C.B. IT-SDN: Improved architecture for SDWSN. In Proceedings of the XXXV Brazilian Symposium on Computer Networks and Distributed Systems, Belem, Brazil, 15–19 May 2017; pp. 15–19.
30. Karakurt, H.B.; Koçak, C. Increasing quality of service in wireless local area networks through fuzzy logic based feedback control method. *Wirel. Netw.* **2023**, *29*, 2217–2233. [CrossRef]
31. Fondo-Ferreiro, P.; Mhiri, S.; López-Bravo, C.; González-Castaño, F.J.; Gil-Castiñeira, F. Fast Decision Algorithms for Efficient Access Point Assignment in SDN-Controlled Wireless Access Networks. *IEEE Trans. Netw. Serv. Manag.* **2019**, *16*, 1059–1070. [CrossRef]
32. Heller, B.; Sherwood, R.; McKeown, N. The controller placement problem. In *First Workshop on Hot Topics in Software Defined Networks*; HotSDN '12; Association for Computing Machinery: New York, NY, USA, 2012; pp. 7–12. [CrossRef]
33. Mudvari, A.; Tassiulas, L. Joint SDN Synchronization and Controller Placement in Wireless Networks using Deep Reinforcement Learning. In *NOMS 2024–2024 IEEE Network Operations and Management Symposium*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–9. [CrossRef]
34. Golshan, R. *LTE Advanced White Paper*; Technical Report; SAI Technology Inc.: Santa Clara, CA, USA, 2011.
35. Limited, A. *Whitepaper: Wi-Fi 7—What Is It All About?* Technical Report; Amdocs Limited: St. Louis, MO, USA, 2023.
36. Coêlho, R.W.; Silva, R.A.; Martimiano, L.A.F.; Leonardo, E.J. IoT and 5G networks: A discussion of SDN, NFV and information security. *J. Braz. Comput. Soc.* **2024**, *30*, 212–227. [CrossRef]
37. Bouhafs, F.; Raschella, A.; Mackay, M.; Hashem Eiza, M.; den Hartog, F. Optimizing Radio Access for Massive IoT in 6G Through Highly Dynamic Cooperative Software-Defined Sharing of Network Resources. *Future Internet* **2024**, *16*, 442. [CrossRef]
38. Yang, D.; Tsai, W.T. Sdn-based congestion control and bandwidth allocation scheme in 5g networks. *Sensors* **2024**, *24*, 749. [CrossRef]
39. Ellawindy, I.; Shah Heydari, S. Crowdsourcing framework for QoE-aware SD-WAN. *Future Internet* **2021**, *13*, 209. [CrossRef]
40. Marwaha, S.; Indulska, J.; Portmann, M. Challenges and recent advances in QoS provisioning in wireless mesh networks. In *2008 8th IEEE International Conference on Computer and Information Technology*; IEEE: Piscataway, NJ, USA, 2008; pp. 618–623. [CrossRef]
41. Meseguer, R.; Molina, C.; Ochoa, S.F.; Santos, R. Energy-aware topology control strategy for human-centric wireless sensor networks. *Sensors* **2014**, *14*, 2619–2643. [CrossRef] [PubMed]
42. Martins, G.C.; Urso, A.; Mansano, A.; Liu, Y.; Serdijn, W.A. Energy-Efficient Low-Power Circuits for Wireless Energy and Data Transfer in IoT Sensor Nodes. *arXiv* **2017**, arXiv:1704.08910. [CrossRef]
43. Santos, R.M.; Orozco, J.; Ochoa, S.F.; Meseguer, R.; Mosse, D. Providing real-time message delivery on opportunistic networks. *IEEE Access* **2018**, *6*, 40696–40712. [CrossRef]
44. Peng, J.; Cheng, L. Revisiting Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA). In *2006 40th Annual Conference on Information Sciences and Systems*; IEEE: Piscataway, NJ, USA, 2006; pp. 1236–1241. [CrossRef]
45. Xiao, X. Chapter 13—QoS in Wireless Networks. In *Technical, Commercial and Regulatory Challenges of QoS*; The Morgan Kaufmann Series in Networking; Xiao, X., Ed.; Morgan Kaufmann: Boston, MA, USA, 2008; pp. 225–246. [CrossRef]
46. Wu, H.; Wang, X.; Zhang, Q.; Shen, X. IEEE 802.11e Enhanced Distributed Channel Access (EDCA) Throughput Analysis. In *2006 IEEE International Conference on Communications*; IEEE: Piscataway, NJ, USA, 2006; Volume 1, pp. 223–228. [CrossRef]

47. Obaidat, I.; Alsmirat, M.; Jararweh, Y. Completing IEEE 802.11e implementation in NS-3. In *2016 7th International Conference on Information and Communication Systems (ICICS)*; IEEE: Piscataway, NJ, USA, 2016; pp. 190–195. [CrossRef]
48. Dolińska, I. The EDCA Implementation in NS-3 Network Simulator. *Vistula Univ. Work. Pap.* **2018**, *59*, 19–29.
49. Li, S.; Hu, D.; Fang, W.; Ma, S.; Chen, C.; Huang, H.; Zhu, Z. Protocol Oblivious Forwarding: Software-Defined Networking with Enhanced Programmability. *IEEE Netw.* **2017**, *31*, 58–66. [CrossRef]
50. Breiman, L. Random Forests. *Mach. Learn.* **2001**, *45*, 5–32. [CrossRef]
51. Salman, H.A.; Ali, K.; Amani, S. Random Forest Algorithm Overview. *Babylon. J. Mach. Learn.* **2024**, *2024*, 69–79. [CrossRef] [PubMed]
52. Mockus, J. *Bayesian Approach to Global Optimization*, 1st ed.; Mathematics and its Applications; Springer: Dordrecht, The Netherlands, 1989; 270p. (In Russian) [CrossRef]
53. Garrido-Merchán, E.; Molina, M.; Martínez, G. Multifidelity Bayesian optimization for hyperparameter tuning of deep reinforcement learning algorithms. *Comput. Artif. Intell.* **2025**, *3*, 2923. [CrossRef]
54. Bergstra, J.; Bardenet, R.; Bengio, Y.; Kégl, B. Algorithms for hyper-parameter optimization. In *25th International Conference on Neural Information Processing Systems*; NIPS'11; Curran Associates Inc.: Red Hook, NY, USA, 2011; pp. 2546–2554.
55. Watanabe, S. Tree-Structured Parzen Estimator: Understanding Its Algorithm Components and Their Roles for Better Empirical Performance. *arXiv* **2025**, arXiv:2304.11127.
56. Pérez, S.; Facchini, H.; Mercado, G.; Bisaro, L.; Campos, J. Throughput quantitative analysis of EDCA 802.11e in different scenarios. *J. Comput. Sci. Technol.* **2013**, *13*, 16–24.
57. Casella, G.; Berger, R.L. *Statistical Inference*, 2nd ed.; Springer: Cham, Switzerland, 2021.
58. NS-3 Project. NS-3: A Discrete-Event Network Simulator. 2026. Available online: https://www.nsnam.org/ (accessed on 15 March 2026).


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.